`DataAnnotations` attributes provide the basic metadata for validation, but no part of listing 7.10 or listing 7.11 uses the validation attributes you added. You still need to add code to read the parameter type's metadata, check whether the data is valid, and return an error response if it's invalid. ASP.NET Core doesn't include a dedicated validation API for that task in minimal APIs, but you can easily add it with a small NuGet package.

7.10.3 Adding a validation filter to your minimal APIs

Microsoft decided not to include any dedicated validation APIs in minimal APIs. By contrast, validation is a built-in core feature of Razor Pages and MVC. Microsoft's reasoning was that the company wanted to provide flexibility and choice for users to add validation in the way that works best for them, but didn't want to affect performance for those who didn't want to use their implementation.

Consequently, validation in minimal APIs typically relies on the filter pipeline. As a classic cross-cutting concern, validation is a good fit for a filter. The only downside is that typically, you need to write your own filter rather than use an existing API. The positive side is that validation gives you complete flexibility, including the ability to use an alternative validation library (such as FluentValidation) if you prefer.

Luckily, Damian Edwards, a project manager architect on the ASP.NET Core team at Microsoft, has a NuGet package called MinimalApis.Extensions that provides the filter for you. Using a simple validation system that hooks into the `DataAnnotations` on your models, this NuGet package provides an extension method called `WithParameterValidation()` that you can add to your